

LAPORAN PRAKTIKUM INDIVIDU
PRAKTIK 1-2
TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum
Mata Kuliah Teknologi IoT II



DISUSUN OLEH :

NAMA	: Raffi Maulana Abdillah
NPM	25782021
PRODI	: TEKNOLOGI REKAYASA INTERNET
KELAS	: 3A
DOSEN	: Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T. Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG 2026

LOGBOOK:

Latihan

1. Pemahaman Logika Pin

Soal: Berdasarkan tabel referensi pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / pull-down), namun menyambungkan tombol tersebut ke pin D8 (GPIO15), lalu menekan tombol tersebut tepat pada detik di mana NodeMCU baru diberi daya?

Jawab: NodeMCU gagal *booting* & program tidak jalan.

Karena, pin D8 tergolong *strapping pin*, dan harus di kondisi LOW saat pertama kali dijalankan agar ESP8266 bisa *booting*. Jika Active-High, tekan tombol membawa pin D8 ke HIGH, membuat ESP8266 eror & sistem gagal *start*.

2. Implementasi Kode

Soal: Modifikasi kode pada Praktikum 1 agar LED berkedip secara tidak simetris, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan 2 baris fungsi `delay()` yang harus Anda ubah.

Jawab:

C++

`const int ledPin = 5;`

`void setup() {`

`Serial.begin(115200);`

`pinMode(ledPin, OUTPUT);`

`Serial.println("Praktikum 1 - Digital Output Dimulai!");`

`}`

`void loop() {`

`digitalWrite(ledPin, HIGH);`

`Serial.println("LED Menyala");`

delay(200);

digitalWrite(ledPin, LOW);

Serial.println("LED Mati");

delay(800);

}

Fungsi delay yang diubah:

- delay(200) di LED Menyala.
- delay(800) di LED Mati.

3. Analisis Sirkuit

Soal: Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami eror kompilasi? Mengapa LED tidak menyala?

Jawab: Program tak mengalami eror kompilasi. Program bisa dikompilasi karena *software* tak menemukan kesalahan rangkaian *hardware*. Karena posisi LED terbalik, arus listrik tersumbat sehingga LED mati.

4. Pengembangan Algoritma

Soal: Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati".

Jawab:

C++

if (buttonState == HIGH) {

digitalWrite(ledPin, LOW);

Serial.println("Tombol ditekan! -> LED OFF");

} else {

digitalWrite(ledPin, HIGH);

Serial.println("Tombol tidak ditekan! -> LED ON");

}

❖ **PRAKTIKUM 1:**

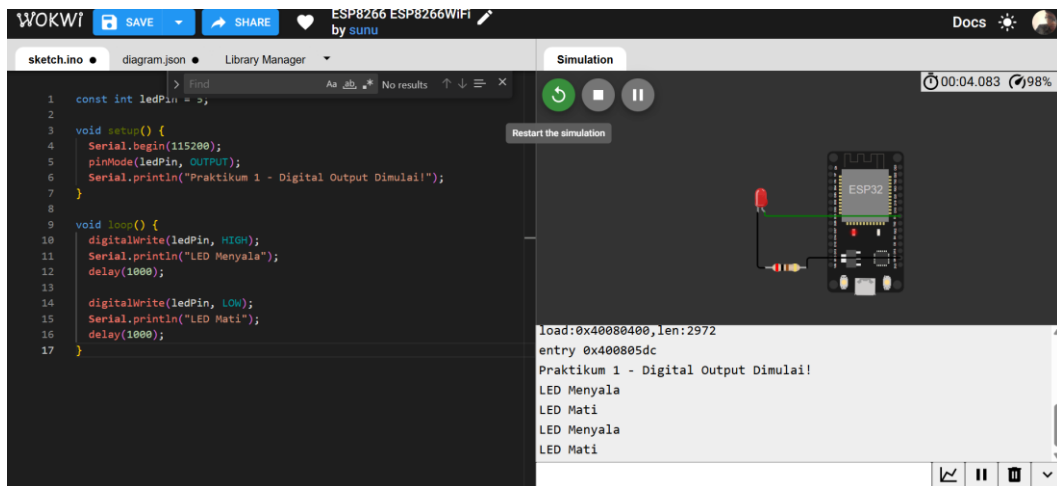
Langkah Kerja Rangkaian:

Pasang NodeMCU pada *breadboard*.

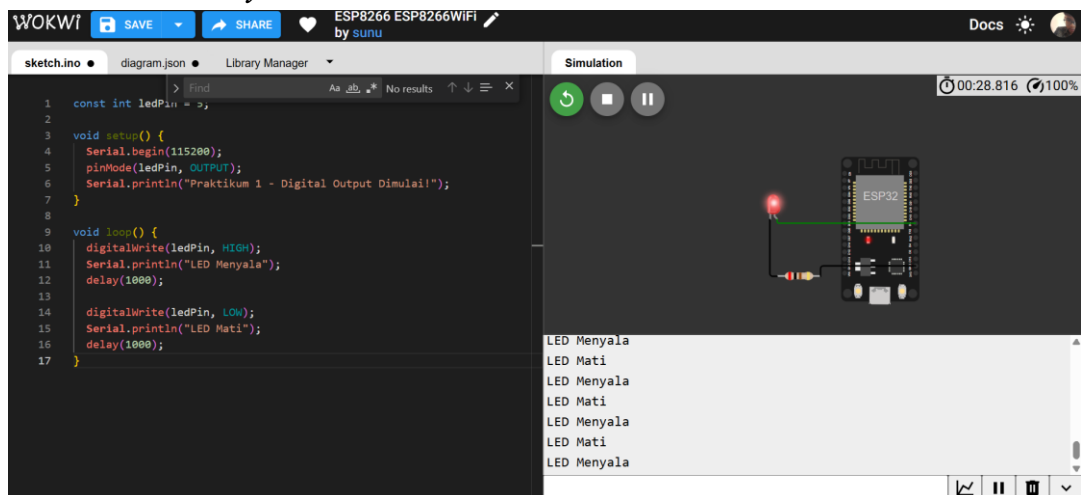
1. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - o Kaki yang **lebih panjang** adalah **Anoda (Positif)**.
 - o Kaki yang **lebih pendek** (dan terdapat sisi datar di tepi plastik kepala LED) adalah **Katoda (Negatif)**.
2. Hubungkan pin **D1 (GPIO 5)** dari NodeMCU ke kaki **Anoda** LED menggunakan kabel *jumper*.
3. Hubungkan kaki **Katoda** LED ke salah satu ujung **Resistor 220 Ohm**.
4. Hubungkan ujung **Resistor** lainnya ke jalur **GND** pada NodeMCU. (Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar).

- SCREENSHOT PRAKTIKUM 1:

- *LED Mati:*



- *LED Nyala:*

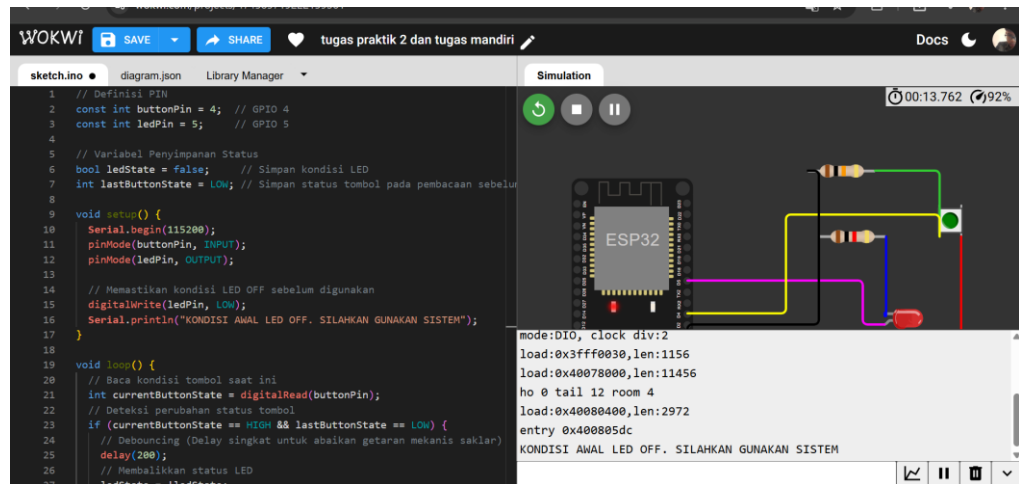


❖ PRAKTIKUM 2:

Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin **D1** dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan **kaki pertama** *push button* ke pin **3V3** (3.3V) pada NodeMCU.
4. Hubungkan **kaki kedua** *push button* ke pin **D2 (GPIO 4)** NodeMCU.
5. Pada **kaki kedua** yang sama, pasang sebuah **Resistor 10k Ohm**, lalu hubungkan ujung lain dari resistor tersebut ke jalur **GND** NodeMCU.

- **SCREENSHOT PRAKTIKUM 2:**



❖ **TUGAS MANDIRI:**

Modifikasi Sistem Sakelar *Toggle (Latching)*.

- Tujuan: Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*.
- Tingkat Kesulitan: Menengah
- Instruksi:
 - i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
 - ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali *lalu dilepas*, LED menyala dan tetap menyala.
 - Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED mati dan tetap mati.

- iii. *Petunjuk*: Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).
- Kriteria Keberhasilan:
 - Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
 - Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
 - Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.

❖ KODE TUGAS MANDIRI:

```
// Definisi PIN
const int buttonPin = 4; // GPIO 4
const int ledPin = 5;    // GPIO 5

// Variabel Penyimpanan Status
bool ledState = false;    // Simpan kondisi LED
int lastButtonState = LOW; // Simpan status tombol pada pembacaan sebelumnya

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);

  // Memastikan kondisi LED OFF sebelum digunakan
  digitalWrite(ledPin, LOW);
  Serial.println("KONDISI AWAL LED OFF. SILAHKAN GUNAKAN SISTEM");
}

void loop() {
  // Baca kondisi tombol saat ini
  int currentButtonState = digitalRead(buttonPin);
  // Deteksi perubahan status tombol
  if (currentButtonState == HIGH && lastButtonState == LOW) {
    // Debouncing (Delay singkat untuk abaikan getaran mekanis saklar)
    delay(200);
    // Membalikkan status LED
    ledState = !ledState;
    // Terapkan status baru ke LED
    digitalWrite(ledPin, ledState ? HIGH : LOW);
  }
}
```

```

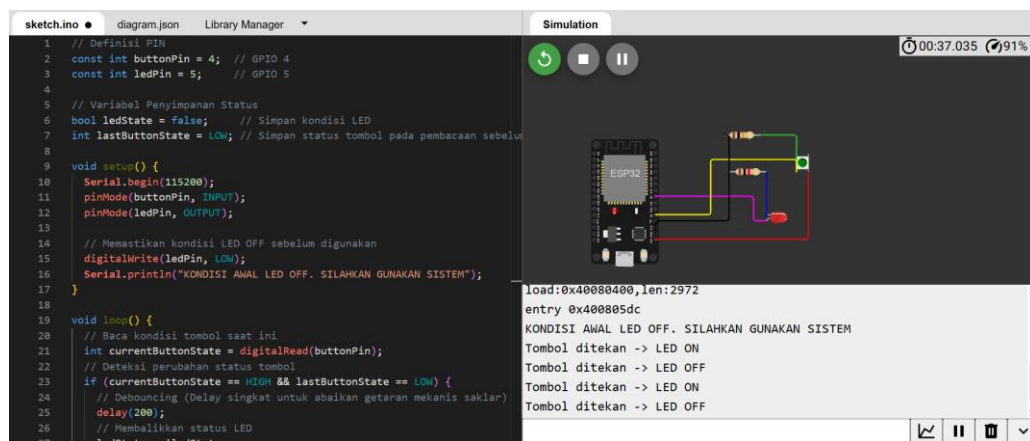
// Tampilkan status ke Serial Monitor
if (ledState) {
    Serial.println("Tombol ditekan -> LED ON");
} else {
    Serial.println("Tombol ditekan -> LED OFF");
}
}

// Simpan kondisi tombol saat ini untuk dibandingkan
lastButtonState = currentButtonState;
}

```

❖ SCREENSHOT TUGAS MANDIRI:

- *LED OFF:*



- *LED ON:*

